

CP(M) 2026: MiniZinc Labs

These labs are not marked. They build towards the Invigilated MiniZinc Assessment on Wednesday 11 November 2026, and Lab 3 (graph burning) is the exercise that assessment will ask about directly.

Each lab has its own folder containing a stub model and a folder of instances:

Lab	Folder	Stub
0. Dominating set	lab-0-dominating-set/	dominating-set.mzn
1. Distance domination	lab-1-distance-domination/	distance-domination.mzn
2. Linear models	lab-2-linear-models/	distance-domination-linear.mzn
3. Graph burning	lab-3-graph-burning/	graph-burning.mzn

Do not rename the decision variables declared in a stub: the output format depends on them, and later labs assume them. You may add further variables.

Input format

All instances describe a finite simple undirected graph $G = (V, E)$ with $V = \{1, \dots, n\}$, in the following format:

```
n = 5;  
m = 6;  
from = [1,1,2,3,3,4];  
to = [2,4,3,4,5,5];
```

Here `n` is the number of vertices, `m` the number of edges, and the e th edge joins `from[e]` and `to[e]`. Each edge is listed once, in one direction only. The example is the graph with edge set $\{12, 14, 23, 34, 35, 45\}$, which we call `small-5` below. Instances for Labs 1 and 2 also contain a line `k = ...`. The first line of each instance file is a comment describing the graph; grids are numbered row by row, starting at the top left.

Running a model

From the MiniZinc IDE, open the stub and select an instance file when prompted. From the command line:

```
minizinc --solver gecode dominating-set.mzn instances/small-5.dzn
```

Useful options are `--solver` (for example `gecode`, `chuffed`, `coin-bc`, `highs`), `--time-limit 30000` (in milliseconds), `-a` (print every solution, or every improving solution), and `--statistics`. All four solvers named here are included with the MiniZinc IDE.

Optimal values for every instance supplied are listed at the end of this sheet, so that you can check your models.

Lab 0: Dominating set

Given a graph $G = (V, E)$, a set $S \subseteq V$ is a *dominating set* if every vertex of G either belongs to S or is adjacent to a vertex of S . The *domination number* $\gamma(G)$ is the size of a smallest dominating set.

Output. The stub declares `array[1..n] of var 0..1: decision`, where `decision[v] = 1` means that $v \in S$. For `small-5`, a correct model prints something like

```
decision = [1, 0, 0, 0, 1];
-----
=====
```

Tasks.

1. Find a minimum dominating set of `small-5` by hand.
2. Complete `dominating-set.mzn` so that it computes a minimum dominating set.
3. Run your model on every instance in `lab-0-dominating-set/instances/` and check the sizes against the table at the end of the sheet.
4. The condition “ v is dominated” can be written with `exists`, or as a sum that must be at least 1. Write both. Do they give the same answers? Is either noticeably faster?
5. Run your model on `grid-8x8.dzn` with Gecode and a time limit of 60 seconds. What is the best solution found, and has the solver proved it optimal? (We return to this instance in Lab 2.)

Extensions.

- An *independent dominating set* is a dominating set in which no two vertices are adjacent. Adapt your model to find a smallest one. On most of the supplied instances its size equals $\gamma(G)$; run it on `double-star` to see a graph where it does not, and explain the difference.
 - A *total dominating set* is a set S in which every vertex of G , including the vertices of S , is adjacent to a vertex of S . Adapt your model. What happens on a graph with an isolated vertex?
-

Lab 1: Distance- k domination

Write $d(u, v)$ for the distance between u and v in G , the number of edges on a shortest path between them. For an integer $k \geq 0$, a set $S \subseteq V$ is a *distance- k dominating set* if every vertex v satisfies $d(v, s) \leq k$ for some $s \in S$. A minimum distance- k dominating set is a smallest such set.

When $k = 1$ this is the dominating set of Lab 0. When $k = 0$ each vertex dominates only itself, so the only distance-0 dominating set is V .

Output. As in Lab 0, the solution is recorded in the 0-1 array `decision`. For `small-5-k2`, a correct model prints

```
| decision = [1, 0, 0, 0, 0];  
| -----  
| =====
```

or another dominating set of size 1.

Tasks.

1. Check by hand that vertex 1 alone is a distance-2 dominating set of `small-5`. Is there a vertex of `small-5` that would not do?
2. Write a model for the fixed case $k = 2$ only, in which you state directly what it means for v to be within distance 2 of a chosen vertex. Test it on `small-5-k2`, `grid-5x5-k2` and `petersen-k2`.
3. Now write a model for general k , read from the data file. The difficulty is that k is not known when you write the model. One approach is to add an auxiliary array of decision variables

```
| array[0..k, 1..n] of var bool: reach;
```

intended to mean that `reach[t, v]` is true exactly when v is within distance t of some chosen vertex. When should `reach[0, v]` be true? Given the values of `reach[t-1, u]` for all u , when should `reach[t, v]` be true? Which entries must be true for `decision` to describe a distance- k dominating set?

4. Test your model on every instance in `lab-1-distance-domination/instances/`, including `path-8-k0`.
5. How many variables and constraints does your model have, as a function of n , m and k ? Compare with the output of

```
| minizinc -c --statistics distance-domination.mzn instances/grid-5x5-k3.dzn
```

which reports the size of the model after MiniZinc has translated it for the solver.

Discussion.

- The distances $d(u, v)$ depend only on the graph, not on the choice of S . Could the model compute them before solving, so that the solver never sees `reach` at all? What would the constraints then look like? (Lab 2 does this.)

- In Task 3, is it enough to require that `reach[t, v]` is true *only if* v is within distance t , rather than *exactly when*? Why might the weaker version be useful?
-

Lab 2: Linear models and MIP solvers

A MiniZinc model can be solved by a mixed-integer programming (MIP) solver such as COIN-BC or HiGHS. These solvers accept only linear equations and inequalities over integer and real variables, so MiniZinc rewrites constructs such as `exists`, `\/` and `<->` into linear form before solving. In this lab you write linear models directly, as you would for an integer linear program, and compare how different solvers behave on them.

The problem is distance- k domination, with the same input and output as Lab 1. The instances are in `lab-2-linear-models/instances/`; the larger ones are intended for the comparisons in Part C.

The stub `distance-domination-linear.mzn` already computes a parameter `nbr`, where `nbr[v]` is the set of neighbours of v . Because `nbr` is computed from the data before solving, it introduces no decision variables.

In Parts A and B, every constraint you state must be a linear equation or inequality over 0-1 variables. You may use `forall` to state one such constraint for each index, and `sum` inside a constraint, but not `exists`, `\/`, `->`, `<->`, `not`, or a comparison such as `(x = 1)` used as a value.

Part A: a linear layered model.

1. Rewrite your Lab 1 model so that `reach` is an array of 0-1 integer variables and every constraint is linear. It may help to answer the second discussion question of Lab 1 first: you need to express “`reach[t, v]` may equal 1 only if v or one of its neighbours has `reach` equal to 1 at step $t - 1$ ” as a single inequality.
2. Check that your model gives the same optimal values as your Lab 1 model on the Lab 1 instances.

Part B: a covering model.

For a vertex v and an integer $r \geq 0$, let $N_r[v] = \{u \in V : d(u, v) \leq r\}$ be the ball of radius r around v . A set S is a distance- k dominating set if and only if $|S \cap N_k[v]| \geq 1$ for every vertex v .

1. Complete the function `ball(v, r)` in the stub so that it returns $N_r[v]$. It should be recursive in r : what is $N_0[v]$, and how is $N_r[v]$ obtained from $N_{r-1}[v]$ and `nbr`? The MiniZinc constructs `if ... then ... else ... endif`, `let { ... } in ...` and `array_union` are useful here.
2. Write the covering model, with one linear constraint for each vertex.
3. How many variables and constraints does this model have, compared with the model of Part A? Where has the work of computing distances gone?

Part C: comparing models and solvers.

1. Run your Lab 1 model and your two models from this lab on `grid-8x8-k1`, `grid-10x10-k2`, `grid-15x15-k2`, `grid-15x15-k3` and `random-50-k2`, using each of `gecode`, `chuffed`, `coin-bc` and `highs`, with a time limit of 30 seconds. Record in a table the best value found, whether optimality was proved, and the time taken. Expect several combinations to reach the time limit without proving optimality.
 2. Which solver is fastest on the grid instances? Can you suggest why? (Consider what a MIP solver learns from the linear relaxation of the covering model, in which each `decision[v]` may take any value in $[0, 1]$.)
 3. Does the choice of model matter more for some solvers than for others?
 4. MiniZinc linearises your Lab 1 model automatically when you run it with `coin-bc`. Compare the output of `--statistics` for that run with the statistics for your Part A model. Is the automatic translation larger or smaller than yours?
-

Lab 3: Graph burning

Graph burning models the spread of a contagion, or of information, through a network. At turn 0 every vertex of G is unburnt. At every turn $t \geq 1$, the following happen in order:

1. every unburnt vertex that is adjacent to a burning vertex becomes burning;
2. the fire-lighter chooses a vertex to set burning.

Burning vertices remain burning. A *burning sequence* is a sequence of choices $(x_1, x_2, \dots, x_b)$ after which every vertex is burning at the end of turn b . The *burning number* $b(G)$ is the length of a shortest burning sequence. It makes no difference to $b(G)$ whether the fire-lighter is allowed to choose a vertex that is already burning, so you may allow this if it is convenient; note, though, that a model which allows it may have many more solutions.

Output. The stub declares

```
| array[1..n] of var 0..n: burn_turn;  
| var 1..n: turns;
```

where `burn_turn[v] = t` if v is chosen at turn t , `burn_turn[v] = 0` if v is never chosen, and `turns` is the number of turns taken. The output item in the stub, which you should not change, prints the number of turns and the burning sequence. For `path-9` a correct model prints something like

```

turns = 3;
sequence = [3, 7, 9];
-----
=====

```

Your model must ensure that exactly one vertex is chosen at each turn $1, \dots, \text{turns}$, and none afterwards.

Tasks.

1. By hand, find a burning sequence of length 3 for the path on 9 vertices, and explain why no sequence of length 2 exists. Then show that the path on 10 vertices needs 4 turns. (You may like to prove that the path on n vertices has burning number $\lceil \sqrt{n} \rceil$.)
2. *Decision version.* Add a parameter `int: T`; and write a model that decides whether G can be burned in exactly T turns. A natural approach is to add an array `burnt[t, v]` of Boolean variables, for $0 \leq t \leq T$, recording whether v is burning at the end of turn t , and to write constraints that simulate the process turn by turn. Test your model on `path-9` with $T = 2$ and $T = 3$.
3. *Optimisation version.* Remove the parameter T and write a model that minimises `turns`. Test it on the smaller instances: `small-5`, `path-9`, `path-10`, `path-16`, `cycle-12`, `petersen`, `binary-tree-15`, `ladder-8`, `grid-4x4` and `grid-5x5`.
4. Check by hand that the sequence your model gives for `small-5` and for `grid-4x4` really does burn the graph. Then run your model on `two-paths-4`, which is not connected. Is the answer correct?
5. *Efficiency.* The remaining instances (`path-30`, `binary-tree-31`, `grid-6x6`, `grid-7x7`, `random-30`, `random-50`) are larger. Find out how long your model takes on each, and then try to make it faster. A straightforward model may take more than a minute on `grid-7x7` and `random-50`. Some directions to consider:
 - *Bounds.* The domain $1..n$ for `turns` is very loose. Show that if G is connected, then $b(G) \leq \text{rad}(G) + 1$, where $\text{rad}(G)$ is the radius of G . Can you use a bound of this kind in your model? Does the bound hold for `two-paths-4`?
 - *Redundancy.* Once `turns` is fixed, how many `burnt` variables does your model need? Are there choices of vertex that can never help, and can you rule them out?
 - *A different model.* Show that $(x_1, \dots, x_b)$ is a burning sequence if and only if every vertex u satisfies $d(u, x_i) \leq b - i$ for some i . Use this characterisation, together with the ideas of Lab 2 Part B, to write a model with no `burnt` array at all. How does it compare with your simulation model?
 - *Solvers.* Compare `gencode`, `chuffed` and a MIP solver on your models.
6. Write a short paragraph (a few sentences) comparing your models: what the variables represent, which constraints rule out an invalid burning sequence, and which model you would choose for a large graph and why. In the invigilated assessment you will be given a sample solution model and asked to explain parts of it in this way, so it is worth practising on models other than your own.

Optimal values

Lab 0: domination number $\gamma(G)$.

Instance	n	$\gamma(G)$
small-5	5	2
path-8	8	3
cycle-5-with-universal	6	1
double-star	8	2
petersen	10	3
binary-tree-15	15	5
grid-5x5	25	7
random-30	30	10
grid-8x8	64	16

Lab 1: size of a minimum distance- k dominating set.

Instance	n	k	Optimum
small-5-k1	5	1	2
small-5-k2	5	2	1
path-8-k0	8	0	8
path-8-k1	8	1	3
path-8-k3	8	3	2
grid-5x5-k1	25	1	7
grid-5x5-k2	25	2	4
grid-5x5-k3	25	3	2
binary-tree-15-k2	15	2	2
binary-tree-15-k3	15	3	1
petersen-k2	10	2	1
cycle-12-k2	12	2	3
random-30-k2	30	2	4
binary-tree-31-k2	31	2	4

Lab 2: size of a minimum distance- k dominating set.

Instance	n	k	Optimum
small-5-k2	5	2	1
path-8-k0	8	0	8
path-8-k3	8	3	2
grid-5x5-k2	25	2	4
petersen-k2	10	2	1
random-30-k2	30	2	4
grid-8x8-k1	64	1	16
grid-10x10-k2	100	2	11
grid-15x15-k2	225	2	22
grid-15x15-k3	225	3	12
random-50-k2	50	2	6

Lab 3: burning number $b(G)$.

Instance	n	$b(G)$
small-5	5	2
path-9	9	3
path-10	10	4
path-16	16	4
cycle-12	12	4
petersen	10	3
binary-tree-15	15	4
ladder-8	16	4
grid-4x4	16	4
grid-5x5	25	4
two-paths-4	8	3
path-30	30	6
binary-tree-31	31	5
grid-6x6	36	5
grid-7x7	49	5
random-30	30	4
random-50	50	4